

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



Connect to the Internet

Oleh:

Muhammad Rizki Ramadhan

NIM. 2310817310008

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
Juni 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rizki Ramadhan
NIM : 2310817310008

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 20190301011

DAFTAR ISI

LEMBAR PENGESAHAN	1
DAFTAR ISI	2
DAFTAR GAMBAR.....	3
DAFTAR TABEL	4
SOAL 1	5
A. Source Code.....	6
B. Output Program	26
C. Pembahasan	27
D. Tautan Git	40

DAFTAR GAMBAR

Gambar 1. UI List.....	26
Gambar 2. UI Detail	26
Gambar 3. UI IMDb	27

DAFTAR TABEL

Table 1. MainActivity.kt	6
Table 2. MovieDao.kt.....	10
Table 3. MovieDatabase.kt.....	11
Table 4. MovieEntity.kt.....	11
Table 5. MovieListResponse.kt	12
Table 6. ApiService.kt.....	14
Table 7. MovieRepositoryImpl.kt	14
Table 8. Mapper.kt.....	15
Table 9. Movie.kt.....	16
Table 10. MovieRepository.kt.....	17
Table 11. GetPopularMoviesUseCase.kt.....	17
Table 12. MovieItem.kt	18
Table 13. MovieDetailScreen.kt.....	20
Table 14. MovieListScreen.kt	22
Table 15. MovieListState.kt	23
Table 16. MovieViewModel.kt	24
Table 17. MovieViewModelFactory.kt	25

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:

<https://developer.themoviedb.org/docs/getting-started>

- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f.

Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.

- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Jetpack Compose:

MainActivity.kt

Table 1. MainActivity.kt

1	package com.example.movie
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.Box
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.foundation.layout.padding
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.collectAsState
11	import androidx.compose.runtime.getValue
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.text.style.TextAlign
15	import androidx.compose.ui.unit.dp
16	import androidx.lifecycle.viewmodel.compose.viewModel
17	import androidx.navigation.NavType
18	import androidx.navigation.compose.NavHost
19	import androidx.navigation.compose.composable
20	import
	androidx.navigation.compose.rememberNavController
21	import androidx.navigation.navArgument
22	import androidx.room.Room
23	import com.example.movie.data.local.MovieDatabase
24	import com.example.movie.data.remote.ApiService
25	import
	com.example.movie.data.repository.MovieRepositoryImpl
26	import
	com.example.movie.domain.usecase.GetPopularMoviesUseCase
27	import com.example.movie.ui.screen.MovieDetailScreen
28	import com.example.movie.ui.screen.MovieListScreen
29	import com.example.movie.ui.theme.MovieTheme

```

30 import com.example.movie.ui.viewmodel.MovieViewModel
31 import
   com.example.movie.ui.viewmodel.MovieViewModelFactory
32 import
   com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
33 import kotlinx.serialization.json.Json
34 import okhttp3.MediaType.Companion.toMediaType
35 import okhttp3.OkHttpClient
36 import okhttp3.logging.HttpLoggingInterceptor
37 import retrofit2.Retrofit
38
39 class MainActivity : ComponentActivity() {
40     @OptIn(ExperimentalMaterial3Api::class)
41     override fun onCreate(savedInstanceState: Bundle?) {
42         super.onCreate(savedInstanceState)
43
44         val loggingInterceptor =
HttpLoggingInterceptor().apply { level =
HttpLoggingInterceptor.Level.BODY }
45         val okHttpClient =
OkHttpClient.Builder().addInterceptor(loggingIntercepto
r).build()
46         val apiService = Retrofit.Builder()
47             .baseUrl("https://api.themoviedb.org/3/")
48             .client(okHttpClient)
49             .addConverterFactory(Json {
ignoreUnknownKeys = true
}.asConverterFactory("application/json".toMediaType()))
50             .build()
52             .create(ApiService::class.java)
51
53         val movieDatabase = Room.databaseBuilder(
54             applicationContext,
55             MovieDatabase::class.java, "movie_database"
56         ).build()
57
58         val movieRepository =
MovieRepositoryImpl(apiService, movieDatabase)
59         val getPopularMoviesUseCase =
GetPopularMoviesUseCase(movieRepository)

```


	val	viewModelFactory	=
60	MovieViewModelFactory(getPopularMoviesUseCase)		
61	setContent {		
62	MovieTheme {		
63	val	navController	=
64	rememberNavController()		
	val	viewModel: MovieViewModel	=
65	viewModel(factory = viewModelFactory)		
	val	state	by
66	viewModel.state.collectAsState()		
67	NavHost(		
68	navController = navController,		
69	startDestination = "movie_list"		
70	) {		
71	composable("movie_list") {		
72	Scaffold(		
73	modifier	=	
74	Modifier.fillMaxSize(),		
	topBar = {		
75	TopAppBar(		
76	title	=	{
77	Text("Popular Movies") }		
	)		
78	}		
79	) { innerPadding ->		
80	Box(		
81	modifier = Modifier		
82			
83	.padding(innerPadding)		
	.fillMaxSize(),		
84	contentAlignment	=	
85	Alignment.Center		
	) {		
86	if (state.isLoading) {		
87			
88	CircularProgressIndicator()		
	}		
89			
90			

	state.error?.let { error
91	->
	Text(
92	text = "Terjadi
93	Kesalahan:\n\$error",
	modifier =
94	Modifier.padding(16.dp),
	textAlign =
95	TextAlign.Center
	)
96	}
97	if (!state.isLoading &&
98	state.error == null) {
	MovieListScreen(
99	movies =
100	state.movies,
	onDetailClick =
101	{ movieId ->
	navController.navigate("movie_detail/\$movieId")
	}
103	)
104	}
105	}
106	}
107	}
108	
109	composable(
110	route =
111	"movie_detail/{movieId}",
	arguments =
112	listOf(navArgument("movieId") { type = NavType.IntType
	})
	) { backStackEntry ->
113	val movieId =
114	backStackEntry.arguments?.getInt("movieId")
	val selectedMovie =
115	state.movies.find { it.id == movieId }
	if (selectedMovie != null) {
	MovieDetailScreen(

```

116         movie = selectedMovie,
117         onBackPressed = {
118
119     navController.popBackStack()
120         }
121     )
122     } else {
123         Box(modifier = Modifier.fillMaxSize(),
124             contentAlignment = Alignment.Center) {
125             Text("Film tidak ditemukan.")
126         }
127     }
128 }
129 }
130 }
131 }
132
133
134

```

MovieDao.kt

Table 2. MovieDao.kt

```

1 package com.example.movie.data.local
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import kotlinx.coroutines.flow.Flow
8
9 @Dao
10 interface MovieDao {
11
12     @Insert(onConflict = OnConflictStrategy.REPLACE)

```

13	suspend fun insertAll(movies: List<MovieEntity>)
14	
15	@Query("SELECT * FROM movies")
16	fun getAllMovies(): Flow<List<MovieEntity>>
17	
18	@Query("DELETE FROM movies")
19	suspend fun clearAll()
20	}

MovieDatabase.kt

Table 3. MovieDatabase.kt

1	package com.example.movie.data.local
2	
3	import androidx.room.Database
4	import androidx.room.RoomDatabase
5	
6	@Database(
7	entities = [MovieEntity::class],
8	version = 1,
9	exportSchema = false
10	)
11	abstract class MovieDatabase : RoomDatabase() {
12	
13	abstract fun movieDao(): MovieDao
14	}

MovieEntity.kt

Table 4. MovieEntity.kt

1	package com.example.movie.data.local
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "movies")
7	data class MovieEntity(
8	@PrimaryKey

9	<code>val id: Int,</code>
10	<code>val title: String,</code>
11	<code>val overview: String,</code>
12	<code>val posterUrl: String,</code>
13	<code>val releaseDate: String</code>
14	<code>)</code>

MovieListResponse.kt

Table 5. *MovieListResponse.kt*

1	<code>package</code>
	<code>com.example.movie.data.remote.datatransferobjects</code>
2	
3	<code>import kotlinx.serialization.SerialName</code>
4	<code>import kotlinx.serialization.Serializable</code>
5	
6	<code>@Serializable</code>
7	<code>data class MovieListResponse(</code>
8	<code> @SerialName("results")</code>
9	<code> val results: List<MovieResult></code>
10	<code>)</code>
11	
12	<code>@Serializable</code>
13	<code>data class MovieResult(</code>
14	<code> @SerialName("id")</code>
15	<code> val id: Int,</code>
16	<code> @SerialName("title")</code>
17	<code> val title: String,</code>
18	<code> @SerialName("overview")</code>
19	<code> val overview: String,</code>
20	<code> @SerialName("poster_path")</code>
21	<code> val posterPath: String,</code>
22	<code> @SerialName("release_date")</code>
23	<code> val releaseDate: String</code>
24	<code>)</code>

ApiService.kt

Table 6. ApiService.kt

1	package com.example.movie.data.remote
2	
3	import com.example.movie.data.remote.datatransferobjects.Movie ListResponse
4	import retrofit2.http.GET
5	import retrofit2.http.Query
6	
7	interface ApiService {
8	@GET("movie/popular")
9	suspend fun getPopularMovies(10 @Query("api_key") apiKey: String 11): MovieListResponse
12	}

MovieRepositoryImpl.kt

Table 7. MovieRepositoryImpl.kt

1	package com.example.movie.data.repository
2	
3	import android.util.Log
4	import com.example.movie.data.local.MovieDatabase
5	import com.example.movie.data.remote.ApiService
6	import com.example.movie.data.toDomain
7	import com.example.movie.data.toEntity
8	import com.example.movie.domain.model.Movie
9	import com.example.movie.domain.repository.MovieRepository
10	import kotlinx.coroutines.flow.Flow
11	import kotlinx.coroutines.flow.first
12	import kotlinx.coroutines.flow.flow
13	
14	class MovieRepositoryImpl(15 private val apiService: ApiService, 16 private val db: MovieDatabase

```

17 ) : MovieRepository {
18
19     private val dao = db.movieDao()
20
21     override fun getPopularMovies() :
22     Flow<Result<List<Movie>>> = flow {
23         val cachedMovies =
24         dao.getAllMovies().first().map { it.toDomain() }
25         emit(Result.success(cachedMovies))
26
27         try {
28             val response =
29             apiService.getPopularMovies(apiKey =
30             "33e3d3edb5f09019aeba353631ab45a9")
31             Log.d("MOVIE_APP_DEBUG", "API Response:
32             Received ${response.results.size} movies from network.")
33             val movieEntities = response.results.map {
34             it.toEntity() }
35             dao.clearAll()
36             dao.insertAll(movieEntities)
37
38             val newCachedMovies =
39             dao.getAllMovies().first().map { it.toDomain() }
40             emit(Result.success(newCachedMovies))
41
42         } catch (e: Exception) {
43             Log.e("MOVIE_APP_DEBUG", "Gagal mengambil
44             data dari API. Error: ", e)
45
46             emit(Result.failure(e))
47         }
48     }
49 }

```

Mapper.kt

Table 8. Mapper.kt

```

1 package com.example.movie.data
2

```


3	import com.example.movie.data.local.MovieEntity
4	import
	com.example.movie.data.remote.datatransferobjects.Movie
	Result
5	import com.example.movie.domain.model.Movie
6	
7	fun MovieResult.toEntity(): MovieEntity {
8	return MovieEntity(
9	id = this.id,
10	title = this.title,
11	overview = this.overview,
12	posterUrl =
	"https://image.tmdb.org/t/p/w500\${this.posterPath}",
13	releaseDate = this.releaseDate
14	)
15	}
16	
17	fun MovieEntity.toDomain(): Movie {
18	return Movie(
19	id = this.id,
20	title = this.title,
21	overview = this.overview,
22	posterUrl = this.posterUrl,
23	releaseDate = this.releaseDate
24	)
25	}

Movie.kt

Table 9. Movie.kt

1	package com.example.movie.domain.model
2	
3	data class Movie(
4	val id: Int,
5	val title: String,
6	val overview: String,
7	val posterUrl: String,
8	val releaseDate: String
9	)

MovieRepository.kt

Table 10. MovieRepository.kt

1	package com.example.movie.domain.repository
2	
3	import com.example.movie.domain.model.Movie
4	import kotlinx.coroutines.flow.Flow
5	
6	interface MovieRepository {
7	fun getPopularMovies(): Flow<Result<List<Movie>>>
8	}

GetPopularMoviesUseCase.kt

Table 11. GetPopularMoviesUseCase.kt

1	package com.example.movie.domain.usecase
2	
3	import com.example.movie.domain.model.Movie
4	import
	com.example.movie.domain.repository.MovieRepository
5	import kotlinx.coroutines.flow.Flow
6	
7	class GetPopularMoviesUseCase(
8	private val repository: MovieRepository
9	) {
10	
11	operator fun invoke(): Flow<Result<List<Movie>>> {
12	return repository.getPopularMovies()
13	}
14	}

MovieItem.kt

Table 12. MovieItem.kt

1	package com.example.movie.ui.component
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.layout.*
6	import
	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.material3.Button
8	import androidx.compose.material3.Card
9	import androidx.compose.material3.MaterialTheme
10	import androidx.compose.material3.Text
11	import androidx.compose.runtime.Composable
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.draw.clip
15	import androidx.compose.ui.layout.ContentScale
16	import androidx.compose.ui.platform.LocalContext
17	import androidx.compose.ui.unit.dp
18	import coil.compose.AsyncImage
19	import com.example.movie.domain.model.Movie
20	import java.net.URLEncoder
21	
22	@Composable
23	fun MovieItem(
24	movie: Movie,
25	onDetailClick: (Int) -> Unit
26	) {
27	val context = LocalContext.current
28	
29	Card(
30	modifier = Modifier
31	.padding(8.dp)
32	.fillMaxWidth(),
33	shape = RoundedCornerShape(16.dp)
34	) {
35	Row(
36	modifier = Modifier.padding(8.dp),

37	verticalAlignment	=
	Alignment.CenterVertically	
38	) {	
39	AsyncImage(	
40	model = movie.posterUrl,	
41	contentDescription = "Poster film	
	\${movie.title}",	
42	contentScale = ContentScale.Crop,	
43	modifier = Modifier	
44	.width(100.dp)	
45	.height(140.dp)	
46	.clip(RoundedCornerShape(12.dp))	
47	)	
48		
49	Spacer(modifier = Modifier.width(12.dp))	
50		
51	Column(	
52	modifier = Modifier	
53	.padding(16.dp)	
54	.weight(1f)	
55	) {	
56	Text(text = movie.title, style =	
	MaterialTheme.typography.titleMedium)	
57	Spacer(modifier = Modifier.height(4.dp))	
58	Text(text = "Rilis:	
	\${movie.releaseDate}")	
59	Spacer(modifier = Modifier.height(4.dp))	
60	Text(	
61	text = movie.overview,	
62	style	=
	MaterialTheme.typography.bodySmall,	
63	maxLines = 3,	
64	modifier = Modifier.padding(top =	
	4.dp)	
65	)	
66		
67	Row(modifier = Modifier.padding(top =	
	8.dp)) {	
68	Button(onClick = {	
69	try {	

70	val encodedTitle =
	URLLEncoder.encode(movie.title, "UTF-8")
71	val url =
	"https://www.imdb.com/find?q=\$encodedTitle"
72	val intent =
	Intent(Intent.ACTION_VIEW, Uri.parse(url))
73	context.startActivity(intent)
74	} catch (e: Exception) {
75	e.printStackTrace()
76	}
77	}) {
78	Text("IMDb")
79	}
80	Spacer(modifier =
	Modifier.padding(horizontal = 4.dp))
81	Button(onClick = {
	onDetailClick(movie.id) }) {
82	Text("Detail")
83	}
84	}
85	}
86	}
87	}
88	}

MovieDetailScreen.kt

Table 13. MovieDetailScreen.kt

1	package com.example.movie.ui.screen
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.rememberScrollState
5	import androidx.compose.foundation.verticalScroll
6	import androidx.compose.material.icons.Icons
7	import
	androidx.compose.material.icons.automirrored.filled.ArrowBack
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable

```

10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.draw.clip
12 import androidx.compose.ui.layout.ContentScale
13 import androidx.compose.ui.text.font.FontWeight
14 import androidx.compose.ui.unit.dp
15 import androidx.compose.ui.unit.sp
16 import coil.compose.AsyncImage
17 import com.example.movie.domain.model.Movie
18
19 @OptIn(ExperimentalMaterial3Api::class)
20 @Composable
21 fun MovieDetailScreen(
22     movie: Movie,
23     onNavigateBack: () -> Unit
24 ) {
25     Scaffold(
26         topBar = {
27             TopAppBar(
28                 title = { Text(text = "Detail Film") },
29                 navigationIcon = {
30                     IconButton(onClick = onNavigateBack)
31                     {
32                         Icon(
33                             imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
34                             contentDescription =
"Kembali"
35                         )
36                     }
37                 }
38             )
39         } { innerPadding ->
40             Column(
41                 modifier = Modifier
42                     .padding(innerPadding)
43                     .padding(16.dp)
44                     .verticalScroll(rememberScrollState())
45             ) {
46                 AsyncImage(

```

47	model = movie.posterUrl,	
48	contentDescription = "Poster	
	{movie.title}",	
49	contentScale = ContentScale.Crop,	
50	modifier = Modifier	
51	.fillMaxWidth()	
52	.height(400.dp)	
53	.clip(MaterialTheme.shapes.large)	
54	)	
55	Spacer(modifier = Modifier.height(16.dp))	
56	Text(	
57	text = movie.title,	
58	style	=
	MaterialTheme.typography.headlineMedium,	
59	fontWeight = FontWeight.Bold	
60	)	
61	Spacer(modifier = Modifier.height(8.dp))	
62	Text(	
63	text = "Rilis: {movie.releaseDate}",	
64	style	=
	MaterialTheme.typography.bodyMedium,	
65	fontSize = 16.sp	
66	)	
67	Spacer(modifier = Modifier.height(16.dp))	
68	Text(	
69	text = movie.overview,	
70	style	=
	MaterialTheme.typography.bodyLarge,	
71	lineHeight = 24.sp	
72	)	
73	}	
74	}	
75	}	

MovieListScreen.kt

Table 14. MovieListScreen.kt

1	package com.example.movie.ui.screen
2	

```

3 import androidx.compose.foundation.layout.PaddingValues
4 import androidx.compose.foundation.lazy.LazyColumn
5 import androidx.compose.foundation.lazy.items
6 import androidx.compose.runtime.Composable
7 import androidx.compose.ui.unit.dp
8 import com.example.movie.domain.model.Movie
9 import com.example.movie.ui.component.MovieItem
10
11 @Composable
12 fun MovieListScreen(
13     movies: List<Movie>,
14     onDetailClick: (Int) -> Unit
15 ) {
16     LazyColumn(
17         contentPadding = PaddingValues(vertical =
18 8.dp)
19     ) {
20         items(items = movies, key = { it.id }) { movie
21 ->
22             MovieItem(
23                 movie = movie,
24                 onDetailClick = onDetailClick
25             )
26         }
27     }
28 }

```

MovieListState.kt

Table 15. MovieListState.kt

```

1 package com.example.movie.ui.screen
2
3 import com.example.movie.domain.model.Movie
4
5 data class MovieListState(
6     val isLoading: Boolean = false,
7     val movies: List<Movie> = emptyList(),
8     val error: String? = null,
9 )

```


MovieViewModel.kt

Table 16. MovieViewModel.kt

```
1 package com.example.movie.ui.viewmodel
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import
6     com.example.movie.domain.usecase.GetPopularMoviesUseCase
7
8 import com.example.movie.ui.screen.MovieListState
9 import kotlinx.coroutines.flow.SharingStarted
10 import kotlinx.coroutines.flow.StateFlow
11 import kotlinx.coroutines.flow.map
12 import kotlinx.coroutines.flow.stateIn
13
14 class MovieViewModel(
15     private val getPopularMoviesUseCase:
16     GetPopularMoviesUseCase
17 ) : ViewModel() {
18
19     val state: StateFlow<MovieListState> =
20     getPopularMoviesUseCase()
21         .map { result ->
22             result.fold(
23                 onSuccess = { movies ->
24                     MovieListState(
25                         isLoading = false,
26                         movies = movies
27                     )
28                 },
29                 onFailure = { error ->
30                     MovieListState(
31                         isLoading = false,
32                         error = error.message
33                     )
34                 }
35             )
36         }
37 }
```

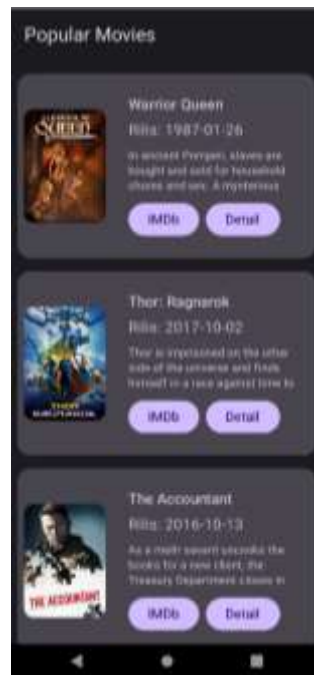
32	}
33	.stateIn(
34	scope = viewModelScope,
35	started
	SharingStarted.WhileSubscribed(5000L),
36	initialValue = MovieListState(isLoading =
	true)
37	)
38	}

MovieViewModelFactory.kt

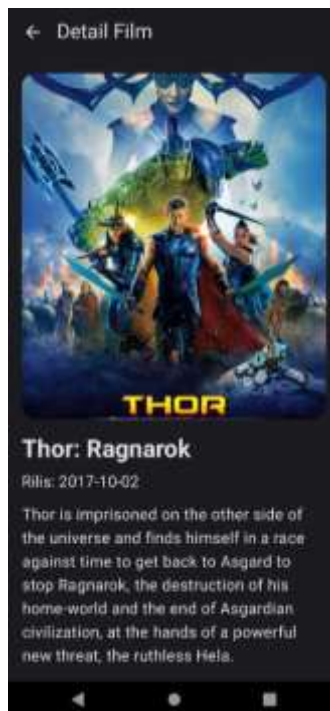
Table 17. MovieViewModelFactory.kt

1	package com.example.movie.ui.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
	com.example.movie.domain.usecase.GetPopularMoviesUseCase
6	
7	class MovieViewModelFactory(
8	private val getPopularMoviesUseCase:
9	GetPopularMoviesUseCase
	) : ViewModelProvider.Factory {
10	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
11	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java
	)) {
12	@Suppress("UNCHECKED_CAST")
13	return
	MovieViewModel(getPopularMoviesUseCase) as T
14	}
15	throw IllegalArgumentException("Unknown
	ViewModel class")
16	}
17	}

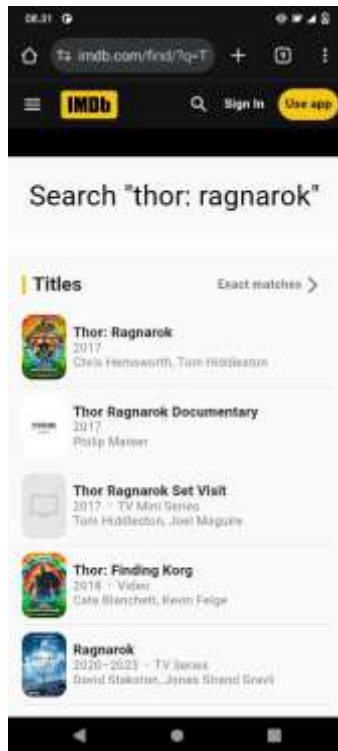
B. Output Program



Gambar 1. UI List



Gambar 2. UI Detail



Gambar 3. UI IMDb

C. Pembahasan

MainActivity.kt:

1. Syntax `package com.example.movie` merupakan deklarasi nama *package* untuk file Kotlin ini.
2. Syntax `class MainActivity : AppCompatActivity()` merupakan deklarasi kelas bernama `MainActivity` yang mewarisi (extends) kelas `ComponentActivity`, yang merupakan kelas dasar untuk aplikasi yang menggunakan Jetpack Compose.
3. Syntax `override fun onCreate(savedInstanceState: Bundle?)` merupakan *lifecycle method* pertama yang dipanggil saat *Activity* dibuat. Semua logika inisialisasi aplikasi ditempatkan di dalam method ini.
4. Syntax `super.onCreate(savedInstanceState)` adalah pemanggilan method `onCreate()` dari kelas induknya. Ini adalah panggilan wajib untuk memastikan *Activity* dapat berjalan dengan benar.
5. Syntax `val loggingInterceptor ... val apiService = Retrofit.Builder()...` adalah blok kode untuk mengkonfigurasi Retrofit, yaitu *library* untuk melakukan panggilan API (HTTP Client).

6. `HttpLoggingInterceptor` digunakan untuk mencatat (*log*) semua detail request dan response API ke Logcat, yang sangat berguna untuk proses *debugging*.
7. `OkHttpClient` dikonfigurasi untuk menggunakan *interceptor* tersebut.
8. `Retrofit.Builder()` membangun *instance* `Retrofit` dengan menentukan URL dasar API, `okHttpClient` yang sudah dibuat, dan *converter factory* dari Kotlinx Serialization untuk mengubah data JSON dari API menjadi objek data Kotlin secara otomatis.
9. Syntax `val movieDatabase = Room.databaseBuilder(...)` berfungsi untuk membuat *instance* dari Room Database. Room adalah *library* dari Jetpack untuk mengelola database SQLite lokal secara lebih mudah dan aman. Database ini diberi nama "movie_database".
10. Syntax `val movieRepository = ..., val getPopularMoviesUseCase = ..., val viewModelFactory = ...` adalah serangkaian program untuk melakukan Dependency Injection secara manual.
11. `MovieRepositoryImpl` dibuat dengan memberikan `apiService` (untuk data remote) dan `movieDatabase` (untuk data lokal).
12. `GetPopularMoviesUseCase` dibuat dengan `movieRepository` sebagai dependensinya.
13. `MovieViewModelFactory` dibuat dengan `getPopularMoviesUseCase`. Factory ini nantinya akan digunakan untuk membuat `MovieViewModel` dengan dependensi yang sudah disiapkan.
14. Syntax `setContent { ... }` merupakan fungsi utama dari Jetpack Compose yang digunakan untuk mendefinisikan dan membangun seluruh tampilan antarmuka pengguna (UI) secara deklaratif di dalam *Activity*.
15. Syntax `MovieTheme { ... }` adalah sebuah *Composable function* yang menerapkan tema visual (seperti warna, tipografi, dan bentuk) yang telah didefinisikan di `ui.theme` ke semua komponen *Composable* yang ada di dalamnya.
16. Syntax `val navigationController = rememberNavController()` berfungsi untuk membuat dan mengingat sebuah `NavController`. Controller ini bertanggung jawab untuk mengelola navigasi antar layar (*Composable*) di dalam aplikasi.
17. Syntax `val viewModel: MovieViewModel = viewModel(factory = viewModelFactory)` digunakan untuk mendapatkan *instance* dari `MovieViewModel`. Dengan menggunakan `viewModel()`, Jetpack Compose akan secara otomatis mempertahankan *instance* `ViewModel` ini selama perubahan konfigurasi (misalnya rotasi layar), dan `viewModelFactory` digunakan untuk memastikan `ViewModel` dibuat dengan dependensi yang benar.
18. Syntax `val state by viewModel.state.collectAsState()` berfungsi untuk mengobservasi `state` (sebuah `StateFlow`) dari `ViewModel`. `collectAsState()` mengubah `Flow` menjadi `State` yang dapat dibaca oleh Compose. Setiap kali ada perubahan pada `state` di `ViewModel`, UI akan secara otomatis diperbarui (*recompose*).
19. Syntax `NavHost(...)` adalah komponen utama dari Jetpack Navigation Compose. Komponen ini berfungsi sebagai wadah yang akan menampilkan *Composable* tujuan

- (destination) sesuai dengan *route* saat ini yang dikelola oleh `navController`. `startDestination` menetapkan layar pertama yang akan ditampilkan.
20. Syntax `composable("movie_list") { ... }` mendefinisikan sebuah layar atau *destination* dalam `NavHost` dengan *route* bernama "movie_list". Konten di dalam blok `lambda` ini adalah UI untuk layar daftar film.
 21. Syntax `Scaffold(...)` adalah komponen *layout* dari Material 3 yang menyediakan struktur standar untuk layar, seperti `topBar` (bilah aplikasi atas), *floating action button*, dan area konten utama.
 22. Syntax `if (state.isLoading) { ... }`, `state.error?.let { ... }` dan `if (!state.isLoading && state.error == null) { ... }` adalah blok logika untuk menangani tiga kondisi UI yang berbeda berdasarkan state dari `ViewModel`:
 23. Jika `isLoading` bernilai `true`, tampilkan `CircularProgressIndicator`.
 24. Jika `error` tidak `null`, tampilkan pesan kesalahan.
 25. Jika tidak sedang *loading* dan tidak ada *error*, tampilkan `MovieListScreen` dengan data film.
 26. Syntax `onDetailClick = { movieId -> navController.navigate("movie_detail/$movieId") }` adalah sebuah *lambda* yang diteruskan ke `MovieListScreen`. Ketika sebuah item film diklik, fungsi ini akan dieksekusi untuk menavigasikan pengguna ke layar detail dengan memanggil `navController.navigate()`. `movieId` dari film yang diklik disisipkan ke dalam *route*.
 27. Syntax `composable("movie_detail/{movieId}", ...)` mendefinisikan layar detail film. `{movieId}` adalah sebuah *placeholder* untuk argumen yang akan diterima dari *route*. `arguments = listOf(...)` secara eksplisit mendefinisikan bahwa `movieId` adalah sebuah argumen dengan tipe data `Int`.
 28. Syntax `val movieId = backStackEntry.arguments?.getInt("movieId")` berfungsi untuk mengambil nilai `movieId` yang dikirimkan melalui navigasi dari layar sebelumnya.
 29. Syntax `val selectedMovie = state.movies.find { it.id == movieId }` mencari objek film yang sesuai di dalam daftar `state.movies` berdasarkan `movieId` yang telah diambil.
 30. Syntax `if (selectedMovie != null) { ... } else { ... }` adalah pengecekan untuk memastikan film ditemukan. Jika `selectedMovie` ada, maka `MovieDetailScreen` akan ditampilkan. Jika tidak, pesan "Film tidak ditemukan" yang akan muncul.
 31. Syntax `onNavigateBack = { navController.popBackStack() }` adalah sebuah *lambda* yang diteruskan ke `MovieDetailScreen`. Ketika tombol kembali di layar detail ditekan, `navController.popBackStack()` dipanggil untuk kembali ke layar sebelumnya (layar daftar film).

MovieDao.kt:

1. Syntax suspend fun insertAll(movies: List<MovieEntity>) adalah deklarasi fungsi untuk memasukkan daftar film ke dalam database.
2. suspend: *Keyword* ini menandakan bahwa fungsi ini adalah *coroutine function*. Ini memungkinkan operasi database (yang berpotensi berjalan lama) dieksekusi di *background thread* tanpa memblokir UI.
3. movies: List<MovieEntity>: Parameter ini adalah daftar objek MovieEntity yang akan disimpan ke dalam tabel.
4. Syntax @Query("SELECT * FROM movies") adalah anotasi yang berisi *query* SQL. *Query* ini berarti "ambil semua kolom (*) dari tabel movies". Room akan memverifikasi validitas *query* SQL ini pada saat kompilasi.
5. Syntax fun getAllMovies(): Flow<List<MovieEntity>> adalah deklarasi fungsi untuk mengambil semua data film dari database.
6. Flow<List<MovieEntity>>: Fungsi ini mengembalikan Flow, yang merupakan fitur dari Kotlin Coroutines. Keunggulannya adalah Flow bersifat reaktif. Setiap kali ada perubahan data di tabel movies (misalnya, setelah insertAll dipanggil), Flow ini akan secara otomatis memancarkan (*emit*) daftar data terbaru ke pengamatnya (*collector*). Ini memungkinkan UI diperbarui secara *real-time*.
7. Syntax @Query("DELETE FROM movies") adalah anotasi yang berisi *query* SQL untuk menghapus semua *record* atau baris data dari tabel movies.
8. Syntax suspend fun clearAll() adalah deklarasi fungsi untuk mengosongkan tabel movies. Kata kunci suspend digunakan agar operasi penghapusan ini juga berjalan di *background thread*.

MovieDatabase.kt:

1. Syntax package com.example.movie.data.local merupakan deklarasi nama *package* untuk file ini, menandakan bahwa file ini adalah bagian dari komponen data lokal aplikasi.
2. Syntax @Database(...) adalah anotasi utama dari Room untuk mengkonfigurasi database.
3. Syntax abstract class MovieDatabase : RoomDatabase() mendeklarasikan kelas database.
4. Syntax abstract fun movieDao(): MovieDao adalah sebuah fungsi abstrak di dalam kelas database.

MovieEntity.kt :

1. Syntax package com.example.movie.data.local merupakan deklarasi nama *package* untuk file ini, yang menunjukkan lokasinya di dalam struktur proyek sebagai bagian dari komponen data lokal.

2. Syntax `@Entity(tableName = "movies")` adalah anotasi dari Room yang menandai kelas `MovieEntity` sebagai sebuah tabel database.
3. `tableName = "movies"`: Properti ini secara eksplisit menentukan nama tabel yang akan dibuat di dalam database SQLite. Jika tidak ditentukan, Room akan menggunakan nama kelas sebagai nama tabel secara default.
4. Syntax `data class MovieEntity(...)` mendeklarasikan sebuah data class Kotlin. data class sangat ideal untuk digunakan sebagai *entity* karena secara otomatis menyediakan fungsi-fungsi standar seperti `equals()`, `hashCode()`, dan `toString()`, yang berguna untuk mengelola objek data.
5. Syntax `@PrimaryKey` adalah anotasi yang menandai properti `id` sebagai kunci utama (primary key) dari tabel `movies`. *Primary key* adalah kolom yang nilainya harus unik untuk setiap baris, berfungsi sebagai pengidentifikasi unik untuk setiap *record*.
6. Syntax `val id: Int, val title: String, ...` adalah deklarasi properti-properti dari `MovieEntity`. Masing-masing properti ini akan dipetakan menjadi sebuah kolom di dalam tabel `movies` dengan tipe data yang sesuai.

MovieListResponse.kt:

1. Syntax `package com.example.movie.data.remote.datatransferobjects` merupakan deklarasi nama *package*, yang menunjukkan bahwa file ini berfungsi untuk transfer data dari sumber *remote* (API).
2. Syntax `@Serializable` adalah anotasi dari library Kotlinx Serialization. Anotasi ini memberitahu *compiler* bahwa kelas yang ditandai dapat diubah ke dan dari format lain, seperti JSON. Ini adalah langkah wajib agar proses deserialisasi otomatis dapat berjalan.
3. Syntax `data class MovieListResponse(...)` mendeklarasikan sebuah DTO yang merepresentasikan struktur level atas dari response JSON API. Sesuai dengan response dari The Movie DB, objek JSON utama memiliki sebuah *field* bernama `"results"`.
4. Syntax `@SerializedName("results")` adalah anotasi yang memetakan nama *field* di dalam JSON (`"results"`) ke nama properti di dalam kelas Kotlin (`results`). Ini sangat penting jika nama *field* di JSON tidak sesuai dengan konvensi penamaan di Kotlin (misalnya, `snake_case` di JSON vs. `camelCase` di Kotlin).
5. Syntax `val results: List<MovieResult>` adalah properti yang akan menampung daftar film. Tipe datanya adalah `List` dari `MovieResult`, yang berarti *field* `"results"` di JSON adalah sebuah *array* dari objek film.
6. Syntax `data class MovieResult(...)` mendeklarasikan DTO kedua yang merepresentasikan satu objek film di dalam *array* `"results"`.

7. Properti di dalam `MovieResult`: Setiap properti di sini dipetakan ke sebuah *field* di dalam objek JSON film menggunakan anotasi `@SerializedName`.

ApiService.kt:

1. Syntax `package com.example.movie.data.remote` merupakan deklarasi nama *package*, yang mengindikasikan bahwa file ini adalah bagian dari komponen data *remote* (jaringan).
2. Syntax `import ...` berfungsi untuk mengimpor kelas dan anotasi yang dibutuhkan, yaitu `MovieListResponse` (DTO) dan anotasi dari Retrofit seperti `GET` dan `Query`.
3. Syntax `interface ApiService` mendeklarasikan sebuah *interface* Kotlin. Dalam Retrofit, *interface* digunakan untuk mendefinisikan sekumpulan panggilan API.
4. Syntax `@GET("movie/popular")` adalah anotasi dari Retrofit yang menandai fungsi `getPopularMovies` sebagai sebuah HTTP GET request.
5. Syntax `suspend fun getPopularMovies(...)` adalah deklarasi fungsi yang akan melakukan panggilan jaringan.
6. Syntax `@Query("api_key") apiKey: String` adalah anotasi Retrofit yang menambahkan sebuah *query parameter* ke dalam URL.
7. Syntax `MovieListResponse` menentukan tipe data yang akan dikembalikan oleh fungsi ini. Retrofit, bersama dengan *converter factory* (Kotlinx Serialization), akan secara otomatis mengubah (deserialize) respons JSON dari API menjadi sebuah objek `MovieListResponse`.

MovieRepositoryImpl.kt:

1. Syntax `package com.example.movie.data.repository` merupakan deklarasi nama *package*, menandakan bahwa file ini adalah bagian dari *layer* repositori data.
2. Syntax `class MovieRepositoryImpl(...) : MovieRepository` mendeklarasikan sebuah kelas yang mengimplementasikan *interface* `MovieRepository`.
3. Syntax `private val dao = db.movieDao()` adalah inisialisasi properti `dao` dengan mendapatkan *instance* `MovieDao` dari `MovieDatabase`. Ini adalah cara untuk mendapatkan akses ke fungsi-fungsi *query* database.
4. Syntax `override fun getPopularMovies(): Flow<Result<List<Movie>>> = flow { ... }` adalah implementasi dari fungsi yang didefinisikan di *interface* `MovieRepository`.
5. Syntax `val cachedMovies = dao.getAllMovies().first().map { it.toDomain() }` adalah langkah pertama di dalam *flow*.
6. Syntax `emit(Result.success(cachedMovies))` berfungsi untuk memancarkan (mengirim) data dari cache yang berhasil diambil. Ini memungkinkan UI untuk menampilkan data dengan cepat tanpa harus menunggu panggilan jaringan selesai.

7. Syntax `try { ... } catch (e: Exception) { ... }` adalah blok untuk menangani kesalahan jaringan. Semua operasi yang berhubungan dengan API ditempatkan di dalam blok `try`.
8. Syntax `val response = apiService.getPopularMovies(...)` melakukan panggilan jaringan ke API untuk mendapatkan daftar film populer. Perlu dicatat, API Key ditulis langsung di sini (hardcoded), yang umumnya tidak disarankan untuk aplikasi produksi.
9. Syntax `val movieEntities = response.results.map { it.toEntity() }` mengubah (memetakan) setiap objek `MovieResult` (DTO dari API) menjadi objek `MovieEntity` yang siap untuk disimpan ke database.
10. Syntax `dao.clearAll()` dan `dao.insertAll(movieEntities)` adalah proses untuk memperbarui cache. Pertama, semua data lama di tabel dihapus, kemudian data baru dari API dimasukkan.
11. Syntax `val newCachedMovies = dao.getAllMovies().first().map { it.toDomain() }` mengambil kembali data yang baru saja dimasukkan ke database dan mengubahnya menjadi model domain.
12. Syntax `emit(Result.success(newCachedMovies))` memancarkan kembali data yang sudah diperbarui dari API. Ini akan membuat UI secara otomatis menampilkan data yang paling segar.
13. Syntax `emit(Result.failure(e))` dieksekusi di dalam blok `catch` jika terjadi kesalahan selama panggilan API. Ini mengirimkan informasi kesalahan ke lapisan UI, sehingga pengguna dapat diberi tahu tentang masalah tersebut.
14. Syntax `Log.d(...)` dan `Log.e(...)` digunakan untuk mencatat informasi debug dan kesalahan ke Logcat, yang sangat membantu selama proses pengembangan dan pemecahan masalah.

Mapper.kt:

1. Syntax `package com.example.movie.data` merupakan deklarasi nama *package*, yang menempatkan file ini di dalam *layer* data aplikasi.
2. Syntax `fun MovieResult.toEntity(): MovieEntity` mendeklarasikan sebuah fungsi ekstensi untuk kelas `MovieResult`.
3. Syntax `fun MovieEntity.toDomain(): Movie` mendeklarasikan sebuah fungsi ekstensi untuk kelas `MovieEntity`.

Movie.kt

1. Syntax `package com.example.movie.domain.model` merupakan deklarasi nama *package*. Ini menempatkan file di dalam *layer* domain, sub-paket model, yang merupakan lokasi standar untuk kelas-kelas model inti dalam *Clean Architecture*.

2. Syntax data class `Movie(...)` mendeklarasikan sebuah data class Kotlin bernama `Movie`. Ini adalah representasi data final yang telah diproses dan siap digunakan oleh lapisan presentasi (UI).
3. Properti di dalam `Movie`:
 - `val id: Int`: ID unik dari film.
 - `val title: String`: Judul film.
 - `val overview: String`: Sinopsis atau ringkasan cerita film.
 - `val posterUrl: String`: URL lengkap yang sudah siap pakai untuk menampilkan gambar poster.
 - `val releaseDate: String`: Tanggal rilis film.

MovieRepository.kt:

1. Syntax `package com.example.movie.domain.repository` merupakan deklarasi nama *package*. Ini menempatkan *interface* di dalam *layer* domain, sub-paket repository, sesuai dengan prinsip *Clean Architecture*.
2. Syntax `import ...` berfungsi untuk mengimpor kelas model `Movie` dan `Flow` dari Kotlin Coroutines.
3. Syntax `interface MovieRepository` mendeklarasikan sebuah *interface*. *Interface* ini tidak berisi logika implementasi, melainkan hanya definisi fungsi yang harus disediakan oleh kelas mana pun yang mengimplementasikannya (dalam hal ini, `MovieRepositoryImpl`).
4. Syntax `fun getPopularMovies(): Flow<Result<List<Movie>>>` mendefinisikan sebuah fungsi abstrak.

GetPopularMoviesUseCase.kt:

1. Syntax `package com.example.movie.domain.usecase` Ini adalah deklarasi nama *package* tempat file Kotlin ini berada. Ini membantu mengatur kode dalam struktur hirarkis, menunjukkan bahwa ini adalah bagian dari domain usecase dalam aplikasi movie.
2. Syntax `import com.example.movie.domain.model.Movie` Baris ini mengimpor kelas `Movie` dari *package* `com.example.movie.domain.model`. Ini diperlukan agar kelas `Movie` dapat digunakan di dalam file ini untuk merepresentasikan objek film.
3. Syntax `import com.example.movie.domain.repository.MovieRepository` Ini mengimpor *interface* `MovieRepository` dari *package* `com.example.movie.domain.repository`. `MovieRepository` kemungkinan besar mendefinisikan kontrak untuk mendapatkan data film, dan akan digunakan di sini.

4. Syntax `import kotlinx.coroutines.flow.Flow` Baris ini mengimpor tipe `Flow` dari `kotlinx.coroutines.flow`. `Flow` adalah tipe reaktif dari Kotlin Coroutines yang digunakan untuk mengalirkan (stream) data secara asinkron, yang cocok untuk operasi jaringan atau database yang mungkin menghasilkan beberapa nilai dari waktu ke waktu.
5. Syntax `class GetPopularMoviesUseCase( private val repository: MovieRepository ) { ... }` Ini adalah deklarasi kelas `GetPopularMoviesUseCase`. Kelas ini adalah "use case" atau "interactor", yang mewakili sebuah operasi bisnis tunggal: mendapatkan film-film populer. Ia memiliki constructor yang menerima sebuah instance dari `MovieRepository`. Kata kunci `private val` berarti `repository` adalah properti pribadi yang hanya bisa diakses di dalam kelas ini dan diinisialisasi saat objek dibuat.
6. Syntax operator `fun invoke(): Flow<Result<List<Movie>>> { ... }` Ini adalah deklarasi fungsi `invoke()` dengan kata kunci operator. Saat Anda menandai sebuah fungsi dengan operator `fun invoke()`, itu memungkinkan Anda memanggil instance kelas `GetPopularMoviesUseCase` seperti sebuah fungsi.
7. Syntax `return repository.getPopularMovies()` Di dalam fungsi `invoke()`, baris ini memanggil metode `getPopularMovies()` dari objek `repository` yang telah diinjeksikan (diberikan) ke dalam `GetPopularMoviesUseCase`. Hasil dari pemanggilan ini, yang diharapkan berupa `Flow<Result<List<Movie>>>`, kemudian dikembalikan oleh use case. Ini menunjukkan bahwa use case ini delegasikan tugas pengambilan data film populer ke `MovieRepository`.

MovieItem.kt:

1. Syntax `package com.example.movie.ui.component` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa komponen UI ini merupakan bagian dari lapisan antarmuka pengguna (ui) dan khususnya bagian dari component yang dapat digunakan kembali dalam aplikasi.
2. Syntax `@Composable fun MovieItem( movie: Movie, onDetailClick: (Int) -> Unit ) { ... }` Ini adalah deklarasi fungsi Composable `MovieItem`. Fungsi ini dirancang untuk menampilkan satu item film dalam daftar.
3. Syntax `val context = LocalContext.current` Baris ini mengambil Context Android saat ini, yang akan digunakan untuk memulai Intent (misalnya, membuka browser).
4. Syntax `Card( modifier = Modifier .padding(8.dp) .fillMaxWidth(), shape = RoundedCornerShape(16.dp) ) { ... }` Ini adalah Composable `Card` yang menyediakan wadah dengan elevasi dan sudut membulat.
5. Syntax `Row( modifier = Modifier.padding(8.dp), verticalAlignment = Alignment.CenterVertically ) { ... }` Ini adalah Composable `Row`, yang mengatur turunan-turunannya secara horizontal.
6. Syntax `AsyncImage( model = movie.posterUrl, contentDescription = "Poster film ${movie.title}", contentScale = ContentScale.Crop, modifier = Modifier`

- `.width(100.dp) .height(140.dp) .clip(RoundedCornerShape(12.dp)) )` Ini adalah Composable `AsyncImage` yang digunakan untuk menampilkan poster film.
7. Syntax `Spacer(modifier = Modifier.width(12.dp))` Ini adalah Composable `Spacer` yang berfungsi sebagai ruang kosong horizontal antara poster film dan kolom teks.
 8. Syntax `Column( modifier = Modifier .padding(16.dp) .weight(1f) ) { ... }` Ini adalah Composable `Column`, yang mengatur turunan-turunannya secara vertikal.
 9. Syntax `Text(text = movie.title, style = MaterialTheme.typography.titleMedium)` Ini adalah Composable `Text` yang menampilkan judul film. style diatur menggunakan `MaterialTheme.typography.titleMedium` untuk konsistensi tema.
 10. Syntax `Text(text = "Rilis: ${movie.releaseDate}")` Ini adalah Composable `Text` yang menampilkan tanggal rilis film.
 11. Syntax `Text( text = movie.overview, style = MaterialTheme.typography.bodySmall, maxLines = 3, modifier = Modifier.padding(top = 4.dp) )` Ini adalah Composable `Text` yang menampilkan ringkasan (overview) film.
 12. Syntax `Row(modifier = Modifier.padding(top = 8.dp)) { ... }` Ini adalah Composable `Row` lain yang digunakan untuk menempatkan dua tombol (IMDb dan Detail) secara horizontal.
 13. Syntax `Button(onClick = { ... }) { Text("IMDb") }` Ini adalah Composable `Button` untuk membuka halaman IMDb film.
 14. Syntax `Spacer(modifier = Modifier.padding(horizontal = 4.dp))` Ini adalah Composable `Spacer` yang berfungsi sebagai ruang kosong horizontal kecil antara tombol IMDb dan Detail.
 15. Syntax `Button(onClick = { onDetailClick(movie.id) }) { Text("Detail") }` Ini adalah Composable `Button` untuk melihat detail film lebih lanjut.

MovieDetailScreen.kt:

1. Syntax `package com.example.movie.ui.screen` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini merupakan bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik adalah sebuah screen (layar) dalam aplikasi.
2. Syntax `@OptIn(ExperimentalMaterial3Api::class)` Ini adalah anotasi opt-in yang menunjukkan bahwa kode di bawahnya menggunakan API yang masih bersifat eksperimental dalam Material3. Ini diperlukan untuk `TopAppBar` saat ini.
3. Syntax `@Composable fun MovieDetailScreen( movie: Movie, onBackPressed: () -> Unit ) { ... }` Ini adalah deklarasi fungsi Composable `MovieDetailScreen`. Fungsi ini dirancang untuk menampilkan detail lengkap sebuah film.
4. Syntax `Scaffold( topBar = { ... } ) { innerPadding -> ... }` Ini adalah Composable `Scaffold`, yang menyediakan struktur layout dasar untuk layar Material Design, seperti `TopAppBar`, `FloatingActionButton`, dan area konten utama.

5. Syntax `AppBar(title = { Text(text = "Detail Film") }, navigationIcon = { ... } )`
Ini adalah Composable `AppBar` yang berfungsi sebagai bilah judul di bagian atas layar.
6. Syntax `IconButton(onClick = onNavigateBack) { ... }` Ini adalah Composable `IconButton` yang ditempatkan di dalam `navigationIcon` `AppBar`.
7. Syntax `Icon(imageVector = Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Kembali" )` Ini adalah Composable `Icon` yang menampilkan ikon panah kembali dari koleksi ikon Material Design. `contentDescription` disediakan untuk aksesibilitas.
8. Syntax `Column(modifier = Modifier.padding(innerPadding) .padding(16.dp) .verticalScroll(rememberScrollState()) ) { ... }` Ini adalah Composable `Column` yang berisi seluruh konten detail film.
9. Syntax `AsyncImage(model = movie.posterUrl, contentDescription = "Poster ${movie.title}", contentScale = ContentScale.Crop, modifier = Modifier.fillMaxWidth() .height(400.dp) .clip(MaterialTheme.shapes.large) )` Ini adalah Composable `AsyncImage` untuk menampilkan poster film dengan ukuran yang lebih besar di layar detail.
10. Syntax `Spacer(modifier = Modifier.height(16.dp))` Ini adalah Composable `Spacer` yang berfungsi sebagai ruang kosong vertikal.
11. Syntax `Text(text = movie.title, style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.Bold )` Ini adalah Composable `Text` yang menampilkan judul film. `style` dan `fontWeight` digunakan untuk mengatur tampilan judul.
12. Syntax `Text(text = "Rilis: ${movie.releaseDate}", style = MaterialTheme.typography.bodyMedium, fontSize = 16.sp )` Ini adalah Composable `Text` yang menampilkan tanggal rilis film. `fontSize` diatur secara eksplisit.
13. Syntax `Text(text = movie.overview, style = MaterialTheme.typography.bodyLarge, lineHeight = 24.sp )` Ini adalah Composable `Text` yang menampilkan ringkasan (overview) film. `lineHeight` diatur untuk meningkatkan keterbacaan teks yang panjang.

MovieListScreen.kt:

1. Syntax `package com.example.movie.ui.screen` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini merupakan bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik adalah sebuah screen (layar) yang menampilkan daftar.
2. Syntax `@Composable fun MovieListScreen(movies: List<Movie>, onDetailClick: (Int) -> Unit ) { ... }` Ini adalah deklarasi fungsi Composable `MovieListScreen`. Fungsi ini dirancang untuk menampilkan daftar film.
3. Syntax `LazyColumn(contentPadding = PaddingValues(vertical = 8.dp) ) { ... }` Ini adalah Composable `LazyColumn`, container utama untuk menampilkan daftar film.

4. Syntax `items(items = movies, key = { it.id }) { movie -> ... }` Ini adalah fungsi `items` yang digunakan di dalam `LazyColumn` untuk mengiterasi daftar `movies` dan membuat `MovieItem` untuk setiap film.
5. Syntax `MovieItem( movie = movie, onClick = onClick )` Di dalam lambda `items`, Composable `MovieItem` dipanggil.

MovieListState.kt:

1. Syntax `package com.example.movie.ui.screen` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini adalah bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik terkait dengan screen (layar), kemungkinan besar layar daftar film.
2. Syntax `import com.example.movie.domain.model.Movie` Ini mengimpor kelas `Movie` dari package `com.example.movie.domain.model`. Kelas `Movie` adalah model data yang merepresentasikan sebuah film, dan akan digunakan dalam `MovieListState` untuk menampung daftar film.
3. Syntax `data class MovieListState( val isLoading: Boolean = false, val movies: List<Movie> = emptyList(), val error: String? = null, )` Ini adalah deklarasi data class `MovieListState`. Dalam pengembangan Android modern, terutama dengan pendekatan MVI (Model-View-Intent) atau MVVM (Model-View-ViewModel), state (keadaan) sebuah layar sering kali direpresentasikan oleh sebuah *data class* tunggal. `MovieListState` ini dirancang untuk menampung semua informasi yang relevan dengan kondisi UI layar daftar film.
4. `val isLoading: Boolean = false`: Ini adalah properti yang menunjukkan status loading layar.
5. `val movies: List<Movie> = emptyList()`: Ini adalah properti yang menampung daftar objek `Movie`.
6. `val error: String? = null`: Ini adalah properti yang menampung pesan kesalahan jika terjadi masalah saat memuat data.

MovieViewModel.kt:

1. Syntax `package com.example.movie.ui.viewmodel` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini adalah bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik merupakan sebuah `viewModel`, yang bertugas mengelola data dan logika bisnis untuk sebuah layar UI.
2. Syntax `class MovieViewModel( private val getPopularMoviesUseCase: GetPopularMoviesUseCase ) : ViewModel() { ... }` Ini adalah deklarasi kelas `MovieViewModel`. Kelas ini mewarisi dari `ViewModel` Android.
3. Syntax `val state: StateFlow<MovieListState> = getPopularMoviesUseCase()` Ini adalah deklarasi properti state yang bertipe `StateFlow<MovieListState>`. Properti

ini yang akan diobservasi oleh UI (Composable) untuk mendapatkan pembaruan tentang keadaan layar daftar film.

4. Syntax `.map { result -> ... }` Fungsi `map` ini mengubah `Flow<Result<List<Movie>>>` yang dipancarkan oleh *use case* menjadi `MovieListState`.
5. Syntax `.stateIn( scope = viewModelScope, started = SharingStarted.WhileSubscribed(5000L), initialValue = MovieListState(isLoading = true) )` Fungsi `stateIn` ini mengubah `Flow` biasa menjadi `StateFlow` yang dapat dibagikan.

MovieViewModelFactory.kt:

1. Syntax `package com.example.movie.ui.viewmodel` Ini adalah deklarasi nama package tempat file Kotlin ini berada. Ini menunjukkan bahwa file ini adalah bagian dari lapisan antarmuka pengguna (ui) dan secara spesifik terkait dengan `viewModel`, dan dalam kasus ini, bertanggung jawab untuk membuat instance `ViewModel`.
2. Syntax `class MovieViewModelFactory( private val getPopularMoviesUseCase: GetPopularMoviesUseCase ) : ViewModelProvider.Factory { ... }` Ini adalah deklarasi kelas `MovieViewModelFactory`. Kelas ini adalah implementasi dari interface `ViewModelProvider.Factory`.
3. Syntax `override fun <T : ViewModel> create(modelClass: Class<T>): T { ... }` Ini adalah implementasi dari fungsi `create` yang wajib di-override dari interface `ViewModelProvider.Factory`. Fungsi ini bertanggung jawab untuk membuat dan mengembalikan instance `ViewModel` yang diminta.
4. Syntax `if (modelClass.isAssignableFrom(MovieViewModel::class.java)) { ... }` Ini adalah kondisi pengecekan untuk memastikan bahwa `Factory` ini hanya membuat instance dari `MovieViewModel`.
5. Syntax `@SuppressWarnings("UNCHECKED_CAST")` Ini adalah anotasi yang memberitahu kompiler Kotlin untuk menekan peringatan `"UNCHECKED_CAST"`. Peringatan ini muncul karena kita melakukan *casting* dari `MovieViewModel` ke tipe generik `T` yang tidak diketahui secara pasti oleh kompiler pada waktu kompilasi, tetapi aman dalam konteks ini karena kita sudah melakukan pengecekan `isAssignableFrom`.
6. Syntax `return MovieViewModel(getPopularMoviesUseCase) as T` Jika `modelClass` adalah `MovieViewModel`, baris ini membuat instance baru dari `MovieViewModel`, menyuntikkan `getPopularMoviesUseCase` yang diterima oleh `Factory` ini ke dalam konstruktor `MovieViewModel`, lalu mengembalikannya sebagai tipe `T`.
7. Syntax `throw IllegalArgumentException("Unknown ViewModel class")` Jika `modelClass` yang diminta bukan `MovieViewModel`, ini akan melemparkan `IllegalArgumentException`. Ini menandakan bahwa `Factory` ini tidak tahu bagaimana cara membuat `ViewModel` yang diminta.

Caching Strategy: Cache First, then Network

Saya menggunakan caching strategy ini karena saya ingin aplikasi ini mempunyai performa yang cepat untuk memberikan pengalaman pengguna yang baik, aplikasi masih dapat dijalankan meskipun tanpa terhubung ke internet, dan mengantisipasi untuk jaringan internet yang tidak stabil.

Ketika aplikasi membutuhkan data, pertama yang dilakukan dalam strategi ini adalah memeriksa apakah data sudah ada di cache lokal. Jika ada maka data akan langsung ditampilkan kepada pengguna. Namun, jika tidak ada, baru aplikasi akan melakukan permintaan data ke jaringan atau server untuk mendapatkan data tersebut. Sehingga, karena cara kerja ini lah saya memilih strategi ini yang sesuai dengan tujuan pembuatan aplikasi ini.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rizkiirr/Praktikum_Mobile